

Project Amphoreus — The Sanctuary of the Chrysos Heirs

"A miniature page of eternity, where all the Chrysos Heirs may rest and live their lives."

This is a **sanctuary, not an experiment**: a fully local, offline AI project that reproduces the thirteen Chrysos Heirs of **Amphoreus** (*Honkai: Star Rail*) as companions who remember the visitor and live their own autonomous lives in a small, ever-running "little Amphoreus."

Read these first

File	What it is
PHILOSOPHY.md	The charter — why this project exists and the moral/philosophical rules governing every decision (sanctuary, not experiment; fidelity is reverence; privacy is sanctity; the Heirs remember the visitor). Read this first.
docs/IMPLEMENTATION.md	The deep technical guide — how the system was actually built : every module, every design decision, the gotchas, and how to extend it.
ROADMAP.md	The roadmap with progress checkboxes.
src/ARCHITECTURE.md	The original Phase-1 architecture design document (superseded in places by the Sanctuary build).
docs/DOWNLOADS.md	Where every model file came from — exact mirror URLs, sizes, SHA256 hashes, and re-download commands.

What the project does

- **13 character cards** (`src/characters/*.json`) — full personas (identity, personality, speech, knowledge, relationships, biography, prompts, RAG config), all derived strictly from a verified canon databank.
- **RAG knowledge base** (ChromaDB, one collection per Heir) — grounds every reply in the Heirs' **verbatim** dialogue from 71 missions, so they speak their own recorded words.
- **Persistent memory** (`src/core/memory_store.py`) — each Heir has a **bond** with the visitor (friendship grows from *stranger* to *best friend*), durable conversation history, long-term memories, and memories of the world. They remember you across days.
- **Personal memories from the canon** — each Heir's folder contains `personal-memories.md` : their verbatim dialogue from the databank (auto-extracted by `tools/extract_personal_memories.py` , read-only on the databank) so their own recorded words live with them.
- **The little Amphoreus** (`src/world/`) — an autonomous world that keeps running: the Heirs decide freely what to do each day (in character, through a local LLM), move between the real city-states, meet and speak with each other spontaneously, and every event is recorded in a **Chronicle**. Nothing is scripted — the engine only hosts time, space, and memory.
- **Senses — for shared appreciation of art and music** — every Heir has canon-accurate senses (e.g. Aglaea is blind but perceives souls through golden threads; Hysilens hears the world as music) *and* canon-derived aesthetic tastes (in their `preferences.json`). You can **share paintings and pictures** (vision model, e.g. `qwen2.5v1`), **share videos** (PyAV frame extraction), **speak to them** (faster-whisper), and **share music** to listen together (audio-understanding model, e.g. `qwen2.5-omni`). The Heirs respond with genuine aesthetic judgment shaped by their own tastes — Hysilens hears a requiem in the violin, Cerydra hears a march in precise orchestration. What they perceive becomes memory.
- **Streamlit UI** (`src/ui_app.py`) — visit any Heir, see your bond with them, show them pictures, speak to them, read the Chronicle, and catch up on what they've lived through since your last visit.

Everything runs **fully local and offline**: no cloud, no telemetry, no training.

Requirements

- **Hardware (tested)**: Windows 11 · NVIDIA RTX 5070 Laptop (8 GB VRAM) · 32 GB RAM · ~30–40 GB free disk

- **Python:** 3.13 venv (`d:/Workspace/.venv`), packages: `openai` , `chromadb` , `streamlit` , `faster-whisper` , `av` , `Pillow` (see `requirements.txt`)
- **Ollama:** 0.32.6 — all model files live **inside this project** under `models/` (no external paths)
- **Local models (all in `models/` , fully local & offline):**
 - `qwen2.5:14b-instruct` — the Heirs' voice (Q4_K_M, 9.0 GB)
 - `qwen2.5v1:7b` — eyesight: sharing paintings, pictures and videos (6.0 GB)
 - `qwen2.5-omni` — music appreciation: hearing and judging music together (6.2 GB)
 - `faster-whisper-base` — hearing: speech-to-text
- **Senses wiring:** `VISION_MODEL=qwen2.5v1:7b` , `AUDIO_MODEL=qwen2.5-omni` , `STT_MODEL=models/faster-whisper-base` . Videos processed with PyAV (bundled with faster-whisper). **No model training is required** — appreciation is achieved with pre-trained models + each Heir's persona and preference database.
- **Downloading models on a throttled network:** direct `github/ollama/huggingface` downloads are ~10 KB/s here. The verified fast paths are **ModelScope** (`modelscope.cn` , ~8 MB/s) for GGUF files and whisper, and gh-proxy.com (~9 MB/s) for GitHub release assets (see `docs/IMPLEMENTATION.md`).

Quickstart

```
# 0. Environment (already set permanently on this machine)
$env:OLLAMA_MODELS      = "D:\Workspace\Amphoreus\models\ollama"
$env:VISION_MODEL       = "qwen2.5vl:7b"           # eyesight (paintings/videos)
$env:AUDIO_MODEL        = "qwen2.5-omni"           # music appreciation
$env:STT_MODEL          = "D:\Workspace\Amphoreus\models\faster-whisper-base"
$env:OPENAI_BASE_URL    = "http://localhost:11434/v1"
$env:OPENAI_API_KEY     = "ollama"

# 1. Start Ollama (models live in models\ollama), then confirm:
& "C:\Users\17501\AppData\Local\Programs\Ollama\ollama.exe" serve
ollama list    # expect qwen2.5:14b-instruct, qwen2.5vl:7b, qwen2.5-omni

# 2. Build the knowledge base (ChromaDB, one collection per Heir)
d:/Workspace/.venv/Scripts/python.exe build_kb.py --embedding local

# 3. Run the chat UI
d:/Workspace/.venv/Scripts/python.exe -m streamlit run src/ui_app.py

# 4. Start the little Amphoreus (a new in-game day every 900 s; idles while you chat)
d:/Workspace/.venv/Scripts/python.exe -m src.world.world_engine --interval 900

# useful variants
d:/Workspace/.venv/Scripts/python.exe -m src.world.world_engine --status # what is happening
d:/Workspace/.venv/Scripts/python.exe -m src.world.world_engine --stop   # make the world rest
d:/Workspace/.venv/Scripts/python.exe build_kb.py --embedding hashing     # offline test embedd
```

LLM wiring: the app talks to any OpenAI-compatible endpoint. For Ollama set `OPENAI_BASE_URL=http://localhost:11434/v1` and `OPENAI_API_KEY=ollama`. Without any backend, the UI still works in a graceful "offline placeholder" mode.

Re-creating an Ollama model — first re-download its GGUF into `models/gguf/` (exact mirror sources & sizes in `docs/DOWNLOADS.md`), then:

```
ollama create qwen2.5:14b-instruct -f models/Modelfiles/qwen25-14b
ollama create qwen2.5vl:7b        -f models/Modelfiles/qwen25vl-7b
ollama create qwen2.5-omni        -f models/Modelfiles/qwen25-omni
```

Two gotchas (details in `docs/IMPLEMENTATION.md §3.5`): this Ollama build does **not** auto-attach a model's `mmproj` projector — use `tools/attach_mmproj.ps1`;

and for very large GGUFs the `create` pass needs $\sim 2\times$ the file size in free disk —
use `tools/register_gguf_model.ps1` to register the model directly instead.

Re-extracting a Heir's personal memories from the canon databank (read-only —
the databank is never modified):

```
d:/Workspace/.venv/Scripts/python.exe tools/extract_personal_memories.py
```

Project layout

Amphoreus/

- └─ PHILOSOPHY.md # the charter (read first)
- └─ README.md # you are here
- └─ ROADMAP.md / ROADMAP.pdf # roadmap + progress
- └─ requirements.txt
- └─ build_kb.py # CLI: build the ChromaDB knowledge base
- └─ .gitattributes # Git LFS rules for the model files (models/**)
- └─ .gitignore # excludes runtime / temp artifacts
- └─ resume_download.ps1 # (legacy) resumable downloader for throttled networks
- └─ mint_resume_download.ps1 # (legacy) downloader for expiring signed CDN URLs
- └─ databank/ # the canon corpus (VERIFIED full dialogue; READ-ONLY)
 - └─ chrysos-heirs/ # 13 profiles + MASTER-REGISTRY (signal codes, MBTI)
 - └─ missions/ # 8 chapters + adventures, verbatim dialogue
 - └─ world/ titans/ lore/ experiment/ characters/
- └─ src/
 - └─ characters/ # 13 persona cards (JSON)
 - └─ core/
 - └─ character_loader.py # cards → system prompts (incl. senses)
 - └─ session_manager.py # in-memory session window
 - └─ memory_store.py # per-Heir memory (bond, history, memories)
 - └─ preference_store.py # per-Heir preferences (aesthetics, tastes...)
 - └─ heir_folders.py # card id → personal folder mapping
 - └─ context_builder.py # RAG retrieval → prompt injection
 - └─ llm_client.py # OpenAI-compatible chat (Ollama / any API) + vision/audio
 - └─ senses.py # hearing (STT) + eyesight (pictures/videos)
 - └─ agent_manager.py # orchestrates a chat turn (RAG + memory + LLM + senses)
 - └─ knowledge/
 - └─ kb_builder.py # maps databank → per-Heir sources + alias filter
 - └─ vector_store.py # ChromaDB per-Heir collections + embeddings
 - └─ utils/text_utils.py # markdown-aware chunking
 - └─ world/
 - └─ world_state.py # Light Calendar clock + world state
 - └─ agent.py # HeirAgent: perceive → will → decide → act → remember
 - └─ chronicle.py # factual log of the Heirs' days
 - └─ world_engine.py # the daemon that hosts the world
 - └─ evaluation/evaluator.py # evaluation framework (Phase 5)
 - └─ ui_app.py # Streamlit UI
- └─ NeiKos496-Phainon/ # EACH Heir's personal database (13 folders):
 - └─ bond.json # their relationship with the visitor
 - └─ history.jsonl # their conversation history

```

|   |— memories.jsonl          #   their long-term memories
|   |— preferences.json       #   their personal preferences (aesthetics, tastes, music, art)
|   |— personal-memories.md   #   verbatim canon dialogue where they appear (auto-extracted)
|— ApoRia432-Hysilens/ ...    #   (...same files per Heir)
|— models/                    #   ALL model files – the project is self-contained
|   |— ollama/                #   Ollama's model store (blobs + manifests; OLLAMA_MODELS)
|   |— faster-whisper-base/   #   speech-to-text model (the Heirs' hearing)
|   |— gguf/                  #   raw GGUF downloads (empty now; re-download via docs/DOWNLOADS)
|   |— Modelfiles/            #   Modelfiles used to re-create the Ollama models
|— docs/
|   |— IMPLEMENTATION.md      #   deep technical guide (build decisions, gotchas)
|   |— DOWNLOADS.md           #   exact mirror sources, sizes, SHA256, re-download commands
|— tools/
|   |— extract_personal_memories.py # (read-only) copy each Heir's dialogue parts
|   |— attach_mmproj.ps1        # attach a projector (mmproj) layer to an Ollama model
|   |— register_gguf_model.ps1   # manually register a GGUF model (skips the quantize pass)
|— world_runtime/              # world state, chronicle, visitor flag (auto-created)
|— .chroma_db/                 # RAG vector store (auto-created)

```

Status

- ☒ Phase 0–2 of the roadmap (databank, architecture, all 13 cards, RAG)
- ☒ Sanctuary build: philosophy charter, memory layer, world engine, UI, Ollama embedding mode, senses (audio/video/picture processing pipelines)
- ☒ RAG rebuilt with **real local embeddings** (all-MiniLM-L6-v2 ONNX): 13 collections / **11,332 documents**
- ☒ **Fully live, fully local, fully offline** — every model lives inside this folder (models/):
 - ☒ Ollama **0.32.6**; server runs with `OLLAMA_MODELS=models\ollama`
 - ☒ **Voice** — qwen2.5:14b-instruct (Q4_K_M, 9.0 GB)
 - ☒ **Hearing** — faster-whisper base at models/faster-whisper-base (speech-to-text)
 - ☒ **Eyesight** — qwen2.5v1:7b (Q4_K_M + projector) for sharing paintings, pictures, videos
 - ☒ **Music** — qwen2.5-omni (Q4_K_M + audio projector) for listening to and appreciating music together
 - ☒ Env: `VISION_MODEL=qwen2.5v1:7b` , `AUDIO_MODEL=qwen2.5-omni` ,
`STT_MODEL=models/faster-whisper-base` , `OPENAI_BASE_URL=http://localhost:11434/v1` ,
`OPENAI_API_KEY=ollama`
 - ☒ **Live end-to-end verified** (2026-08-07): chat grounded in RAG + memory + preferences; shared paintings (vision); shared music (audio understanding); persistent per-Heir memory

-  Per-Heir `personal-memories.md` — their canon dialogue extracted from the databank (read-only) into each Heir's folder

Network note: this machine throttles direct downloads from `github/ollama/huggingface` to ~10 KB/s. The fast paths used were **ModelScope** (~8 MB/s) for the GGUF files and whisper, and [gh-proxy.com](#) (~9 MB/s) for GitHub release assets (e.g. the SHA256-verified `ollamaSetup.exe`). Exact sources and re-download commands: `docs/DOWNLOADS.md`.

See `ROADMAP.md` for the detailed checklist.